

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра інформаційних систем та технологій

Спеціальність 126 Інформаційні системи та технології

ЗВІТ

з переддипломної практики бакалаврів
на тему:

“ Інформаційна система аналізу текстових документів та пошуку схожих матеріалів ”

Місце проходження практики: Науково-дослідна лабораторія вбудованих систем та інтернету речей НТУУ КПІ ім. Ігоря Сікорського, ФІОТ в м. Київ

Виконав студент

ІС-22 Слива Денис Олександрович
(шифр групи, прізвище, ім'я, по батькові)


(підпис)

Керівник практики
від університету

Жураковська Оксана Сергіївна
(прізвище, ім'я, по батькові)

(підпис)

Керівник практики
від підприємства

Мягкий Михайло Юрійович
(прізвище, ім'я, по батькові)


(підпис)

Дата захисту

Оцінка

„_____”

Київ 2026

ЗМІСТ

ПЕРЕЛІК СКОРОЧЕНЬ.....	3
ВСТУП.....	4
1 ОПИС ПРЕДМЕТНОЇ ОБЛАСТІ.....	6
1.1 Опис процесу діяльності.....	7
1.2 Постановка задачі.....	8
1.2.1 Призначення системи.....	9
1.2.2 Цілі та задачі розробки.....	10
Висновок до розділу 1.....	11
2 АНАЛІЗ ІСНУЮЧИХ РІШЕНЬ.....	12
2.1 Програмне рішення «Uniqecheck».....	13
2.2 Програмне рішення «Plagiarismcheck.org».....	14
2.3 Програмне рішення «Grammarly».....	16
Висновок до розділу 2.....	18
3 ПРОЄКТУВАННЯ АРХІТЕКТУРИ СИСТЕМИ.....	19
3.1 Формулювання вимог до системи.....	20
3.1.1 Функціональні вимоги.....	21
3.1.2 Нефункціональні вимоги.....	21
3.2 Вибір технологій розробки.....	22
3.2.1 Система управління базами даних.....	23
3.2.2 Серверна частина.....	24
3.2.3 Модель векторного перетворення тексту.....	25
3.2.3 Клієнтська частина.....	27
Висновок до розділу 3.....	28
4 МАТЕМАТИЧНЕ ЗАБЕЗПЕЧЕННЯ СИСТЕМИ.....	28
4.1 Лексичне порівняння.....	29
4.2 Структурне порівняння.....	31
4.3 Семантичне порівняння.....	33
Висновок до розділу 4.....	35
5 РОЗРОБЛЕННЯ ІНФОРМАЦІЙНОЇ СИСТЕМИ.....	35
5.1 Створення структури бази даних.....	35
5.2 Розроблення серверної частини.....	39
5.3 Розроблення веб-інтерфейсу користувача.....	47
Висновок до розділу 5.....	50
ВИСНОВКИ.....	52
ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	54

ПЕРЕЛІК СКОРОЧЕНЬ

TF-IDF - Term Frequency-Inverse Document Frequency

REST - Representational State Transfer

API - Application Programming Interface

HTTP - HyperText Transfer Protocol

ВСТУП

В інформаційному суспільстві, яке прийшло на зміну постіндустріальному, інформація та знання стали основним продуктом виробництва і це кардинально змінило спосіб життя людей та суспільно-політичний устрій країн. Обсяг інформації в такому суспільстві зростає експоненційно, подвоюючись приблизно кожні два роки. Кожного дня в інтернеті публікуються нові наукові статті, навчальні матеріали, корпоративні документи та інші види текстів, що формують величезні масиви даних. У таких умовах стає все складніше швидко знаходити потрібні матеріали та перевіряти їх на унікальність. Це створює потребу в системі, яка здатна не тільки зберігати тексти, а й аналізувати їх зміст та визначати ступінь подібності двох текстових документів.

Майже всі інструменти пошуку працюють, використовуючи ключові слова. Таким чином, можна легко знайти документ, що містить потрібні слова, але в той же час неможливо зрозуміти, наскільки цей текст близький за змістом. Через це звичайний користувач часто отримує десятки нерелевантних результатів та має витратити тривалий час на порівняння вручну. Наприклад, студенту чи викладачу буде складно перевірити роботу на плагіат, а науковцю – знайти схожі дослідження.

Розроблювана інформаційна система вирішує цю проблему, пропонуючи інший підхід. Вона дозволяє завантажити документ, обробити його та перетворити у зручне для подальшого аналізу векторне представлення, яке легко порівняти за змістом з іншим векторним представленням. У результаті користувач отримує список найбільш релевантних документів, таких, що мають найбільшу схожість з завантаженим користувачем документом. Цей процес робить пошук значно точнішим та зручнішим.

Застосування цієї системи можна знайти в багатьох сферах. Вона може бути використана школами та університетами задля забезпечення принципів академічної доброчесності, науковими установами – задля знаходження схожих публікацій за тією ж темою, приватними компаніями – задля збільшення ефективності при роботі з документами.

Отже, розробка інформаційної системи аналізу текстових документів та пошуку схожих матеріалів є актуальним завданням, враховуючи сучасні тенденції розвитку цифрових технологій, які в найближчому майбутньому будуть тільки посилюватись. Така система допомагатиме впорядкувати великі обсяги інформації, економлячи час користувача та пропонуючи простіший спосіб роботи з цією інформацією. У підсумку, це дозволить зробити взаємодію з текстовими документами більш швидкою, зручною та ефективною, що підкреслює важливість створення такої системи.

1 ОПИС ПРЕДМЕТНОЇ ОБЛАСТІ

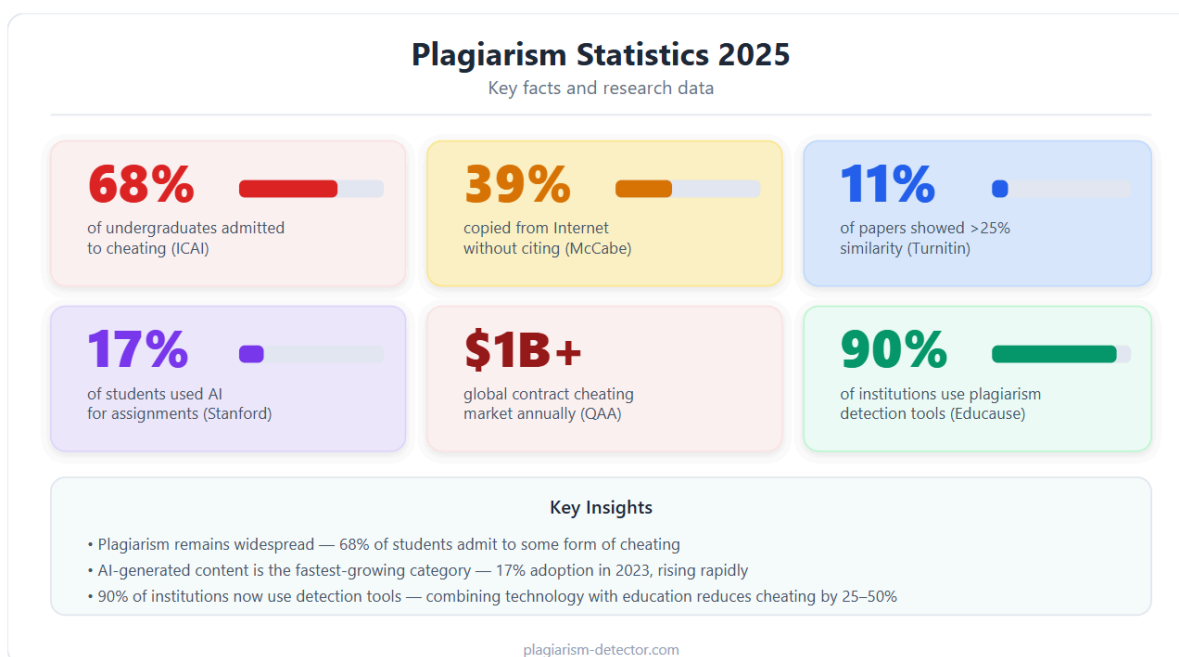


Рисунок 1.1 – Статистика плагіату по даним сайту
plagiarism-detector.com [1]

Плагіат є глобальною проблемою, яка охоплює всі сфери, де створюється текстовий контент – від освіти та науки до журналістики та бізнесу. За даними досліджень International Center for Academic Integrity (ICAI), приблизно 68% студентів бакалаврату зізнаються у використанні різних форм академічної недоброчесності, включно з плагіатом, протягом свого навчання. Подібні результати залишаються стабільними протягом десятиліть досліджень.

Метааналіз, опублікований у журналі PLOS ONE, показує, що в середньому близько 30% студентів хоча б один раз вдавалися до плагіату. При цьому рівень порушень суттєво відрізняється залежно від країни та може коливатися від менш ніж 10% до понад 50%, що пов'язано з культурними особливостями, рівнем контролю та обізнаністю щодо академічної доброчесності.

Проблема плагіату виходить далеко за межі освіти. Згідно зі звітом компанії iThenticate (Turnitin), приблизно 1 із 6 наукових рукописів, поданих до журналів, містить значні текстові збіги з раніше опублікованими джерелами. У сфері журналістики та медіа також регулярно фіксуються випадки плагіату, що підривають довіру до інформаційних ресурсів.

Додаткові дослідження підтверджують масштабність проблеми: за даними Turnitin, близько 11% студентських робіт містять суттєві збіги тексту (понад 25% схожості), а дослідження Bretag та співавторів показало, що понад 6% студентів вдавалися до повного замовлення або купівлі робіт. Навіть у науковому середовищі фіксуються випадки академічних порушень, зокрема плагіату та фальсифікації даних, які розслідуються такими організаціями, як Office of Research Integrity (ORI).

У сукупності ці дані демонструють, що проблема плагіату є системною та глобальною, а не поодинокую. Вона негативно впливає на якість освіти, довіру до наукових результатів і репутацію організацій.

Саме тому в сучасних умовах особливо актуальною є розробка ефективних автоматизованих інструментів для перевірки текстів на унікальність. Такі системи дозволяють своєчасно виявляти запозичення, підвищувати рівень академічної доброчесності та забезпечувати об'єктивну оцінку текстових матеріалів.

1.1 Опис процесу діяльності

Процес розробки інформаційної системи аналізу текстових документів та пошуку схожих матеріалів має виконувати наступне завдання: допомагати користувачеві швидко та ефективно перевірити документ на унікальність та знайти близькі за змістом матеріали. Для цього потрібно спочатку створити базу даних та заповнити її документами.

Цей крок є надзвичайно важливим, тому що дозволяє зберігати та накопичувати матеріали для подальшого аналізу та порівняння.

Далі переходимо до другого кроку, а саме взаємодії користувача з системою. Коли користувач завантажує новий текстовий документ, то система має виконувати його попередню обробку (очищення, токенізацію, нормалізацію) та перетворення в числовий вектор, який відображає зміст текстового документу. Завдяки цьому матеріали можуть бути порівняні не лише за словами, а й за змістовим наповненням, що робить пошук точнішим.

Потім переходимо до третього кроку, а саме пошуку схожих матеріалів. Числове представлення документу користувача порівнюється з усіма числовими представленнями, що вже є у базі даних, і система визначає ступінь їх схожості. Текстові документи, що мають найбільшу схожість із завантаженим документом, виводяться користувачу в форматі списку, який дає уявлення про те, наскільки унікальним є завантажений документ та чи є схожі на нього роботи серед представлених.

Підсумовуючи, система працює за принципами клієнт-серверної архітектури. Серверна частина(back-end) відповідає за обробку запитів та взаємодію з базою даних, а клієнтська частина(front-end) дає можливість взаємодіяти з серверною частиною за допомогою простого та зрозумілого інтерфейсу. В результаті, користувач має зручний інструмент, що дозволяє йому перевірити документ на схожість з іншими документами, які присутні в базі даних. Отриману інформацію можна використати для подальшого аналізу.

1.2 Постановка задачі

Інформаційна система аналізу текстових документів та пошуку схожих матеріалів має пропонувати користувачу інструментарій як для

зберігання текстових документів, так і для подальшого аналізу їх змісту та порівняння з іншими матеріалами, щоб знаходити тематично схожі роботи. Дана система має бути корисною для якнайбільшої кількості користувачів: від студентів та викладачів вищих навчальних закладів, що перевіряють роботи на унікальність, до науковців, що шукають матеріали, дотичні до теми їх дослідження.

Виходячи з цього, можемо сформулювати такі задачі, які наша система має виконувати:

- можливість завантаження текстових документів у форматі PDF;
- автоматична обробка тексту та перетворення його в форму, зручну для аналізу;
- семантичний пошук схожих текстових документів;
- виведення списку найбільш релевантних результатів пошуку із зазначенням ступеня схожості;
- масштабованість системи та можливість працювати з великими обсягами даних.

Підсумовуючи, наша система повинна автоматизувати процес пошуку схожих матеріалів, зробивши його точним та доступним для різних категорій користувачів.

1.2.1 Призначення системи

Інформаційна система аналізу текстових документів та пошуку схожих матеріалів автоматизує такий вид діяльності, як аналіз текстових документів з метою визначення їх унікальності та пошуку матеріалів, близьких за змістом до обраного текстового документу. Процеси, які автоматизуються в ході роботи: зберігання, обробка та порівняння текстових документів.

Об'єктами автоматизації в даному випадку виступають:

- текстові документи, що завантажуються користувачем;
- база даних текстових документів, де вони зберігаються та поступово накопичуються;
- попередня обробка тексту (очищення, токенізація, нормалізація);
- перетворення тексту у числовий вектор;
- семантичний пошук, що визначає ступінь схожості між документами;
- інтерфейс для взаємодії користувача із системою.

Сферами діяльності, в яких може бути використана дана система, є:

1. Освіта: для перевірки на унікальність робіт студентів та дотримання принципів академічної доброчесності;
2. Наука: для пошуку схожих досліджень;
3. Бізнес: оптимізація роботи з великими обсягами інформації.

Таким чином, призначення системи полягає в тому, щоб розробити зручний та ефективний інструмент для автоматизованого аналізу текстових документів та пошуку семантично близьких документів серед всіх збережених до бази даних. Автоматизація цього процесу має на меті зробити роботу з текстовою інформацією швидшою та результативнішою.

1.2.2 Цілі та задачі розробки

Мета розробки полягає у спрощенні аналізу текстових документів та пошуку схожих матеріалів шляхом автоматизації цього процесу. Реалізація такої системи дозволить зменшити витрати часу користувачів на порівняння текстів, а також оптимізує роботу з великими масивами даних.

Для досягнення поставленої мети потрібно виконати такі задачі:

- провести аналіз існуючих методів пошуку схожих матеріалів та визначити їх переваги та недоліки;

- розробити структуру бази даних для зберігання текстових документів;
- розробити алгоритм перетворення тексту у числовий вектор;
- створити модуль пошуку, що визначає ступінь схожості між документами;
- оптимізувати процес пошуку для роботи з великими масивами даних, забезпечивши масштабованість системи;
- створити клієнтський модуль з інтерфейсом для взаємодії користувача з системою;
- провести тестування системи та оцінити її ефективність та продуктивність.

Таким чином, виконання зазначених задач дозволить досягти поставленої мети, а саме - створити інформаційну систему, що забезпечить для користувачів можливість здійснювати аналіз текстових документів та пошук схожих матеріалів автоматизовано.

Висновок до розділу 1

У даному розділі було детально проаналізовано предметну область та описано процеси, що лежать в основі роботи інформаційної системи аналізу текстових документів та пошуку схожих матеріалів, а саме: створення бази даних текстових матеріалів, їх обробка, перетворення у числовий вектор, а також визначення ступеня схожості між документами.

У межах постановки задачі було визначено інструментарій системи та основні задачі, які має виконувати система. Окремо описано призначення системи та визначено об'єкти автоматизації, такі як текстові документи, база даних, процеси попередньої обробки тексту, перетворення його у числовий вектор та семантичний пошук. Також окреслено сфери

діяльності, в яких може бути використана дана система, зокрема освіта, наука та бізнес.

Було сформульовано мету розробки та перелік задач, потрібних для її досягнення, що включають аналіз існуючих методів вирішення проблеми, проєктування структури бази даних, розробку алгоритму пошуку, а також створення інтерфейсу та тестування системи.

Отже, у першому розділі дипломної роботи сформовано теоретичні засади майбутньої розробки. Отримані результати визначають напрям подальшої роботи та формують основу для переходу до більш конкретних етапів, таких як проєктування архітектури програмного забезпечення, вибору технологій розробки та безпосередньої розробки системи у наступних розділах.

2 АНАЛІЗ ІСНУЮЧИХ РІШЕНЬ

Для розробки інформаційної системи аналізу текстових документів та пошуку схожих матеріалів необхідно спочатку зробити огляд вже існуючих рішень, що виконують перевірку текстових документів на унікальність та визначають ступінь їх схожості. Це потрібно для того, щоб мати загальне уявлення про прогрес в даному напрямку на світовому ринку та розуміти, чим саме наша інформаційна система буде краща за аналогічні системи.

Подібні сервіси широко застосовуються у сферах освіти, науки та бізнесу задля дотримання принципів академічної доброчесності шляхом перевірки робіт на унікальність, а також задля оптимізації роботи з великими масивами інформації. Серед всіх існуючих рішень було обрано три найкращі, аналіз яких дозволить нам визначити сильні та слабкі сторони такого типу систем та обґрунтувати доцільність розробки власного продукту в цьому сегменті: Uniqecheck, PlagiarismCheck.org та Grammarly.

2.1 Програмне рішення «Uniqecheck»

Uniqecheck – онлайн-сервіс для перевірки на плагіат, який працює на базі передових AI-алгоритмів, які ретельно аналізують кожен текст. Система здатна виявляти навіть незначні збіги, включаючи перефразування, заміну слів і зміну структури речень [1].

Принцип роботи сервісу інтуїтивно зрозумілий і не потребує складних налаштувань. Користувач може завантажити документ або вставити текст безпосередньо в поле перевірки. Цей функціонал зображено на рисунку 2.1.

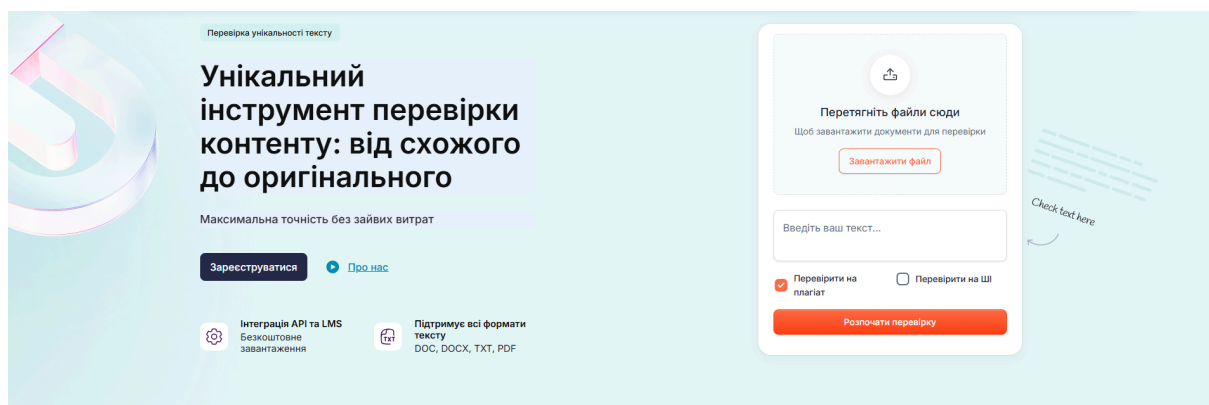


Рисунок 2.1 – Меню завантаження текстового документу в системі Uniqecheck

Після завантаження текстового документу користувач може натиснути на кнопку «Розпочати перевірку», що дозволить інструменту проаналізувати текст, порівнюючи його з веб-сторінками, академічними журналами, науковими роботами та іншими публікаціями. Після запуску аналізу система швидко формує звіт, у якому виділені проблемні фрагменти та надані рекомендації для покращення тексту. Цей функціонал зображено на рисунку 2.2.

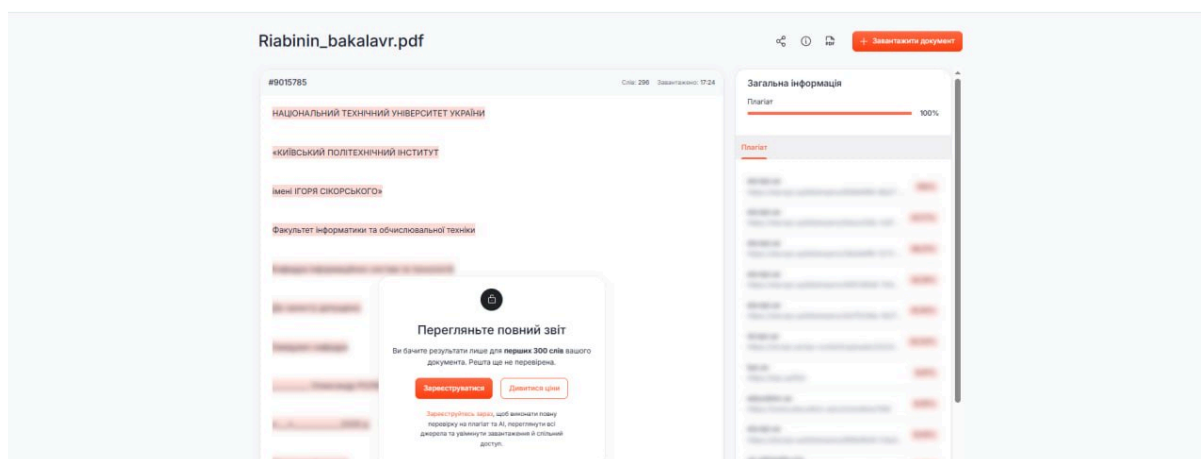


Рисунок 2.2 – Меню звіту в системі Uniqecheck

Для того, щоб виконати повну перевірку на плагіат, переглянути всі джерела та увімкнути завантаження й спільний доступ, потрібно оформити підписку на даний сервіс, що для звичайного користувача є великим мінусом.

Загалом, онлайн-сервіс Uniqecheck має багато сильних сторін, таких як простий інтерфейс, швидка перевірка текстів, підтримка різних форматів документів. Окрім цього, сервіс ще й використовує власну базу даних джерел і може визначати контент, створений штучним інтелектом.

В той же час, сервіс має і суттєві обмеження. Семантичний аналіз тут реалізований на базовому рівні, а також відсутній аналіз фрагментів. Більше того, повний доступ до функціоналу можливий лише за умови оформлення платної підписки, що негативно впливає на досвід роботи з продуктом.

2.2 Програмне рішення «Plagiarismcheck.org»

Plagiarismcheck.org – це сервіс для перевірки на плагіат із зазначенням відсотка, який може допомогти вчителям та студентам оцінити оригінальність будь-якого тексту онлайн [2].

Принцип роботи сервісу такий же, як і в попереднього інструменту. Функціонал завантаження текстового документу зображено на рисунку 2.3.

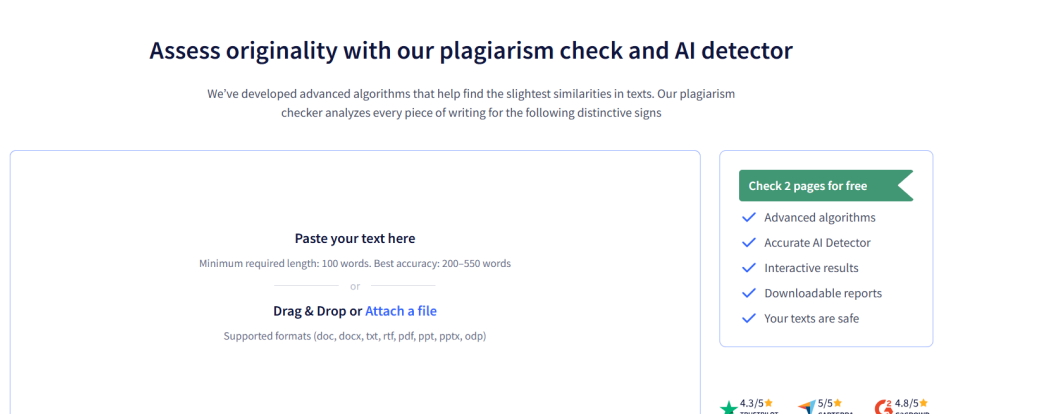


Рисунок 2.3 – Меню завантаження текстового документу в системі Plagiarismcheck.org

Після завантаження текстового документу користувач може натиснути на кнопку «Check text», що запустить процес аналізу. В результаті користувач отримає звіт, в якому вказано відсоток подібності, а також відсоток ідентичного тексту та зміненого. Окрім цього, можна побачити цитати та посилання, які були використані в завантаженому документі. Функціонал звіту зображено на рисунку 2.4.

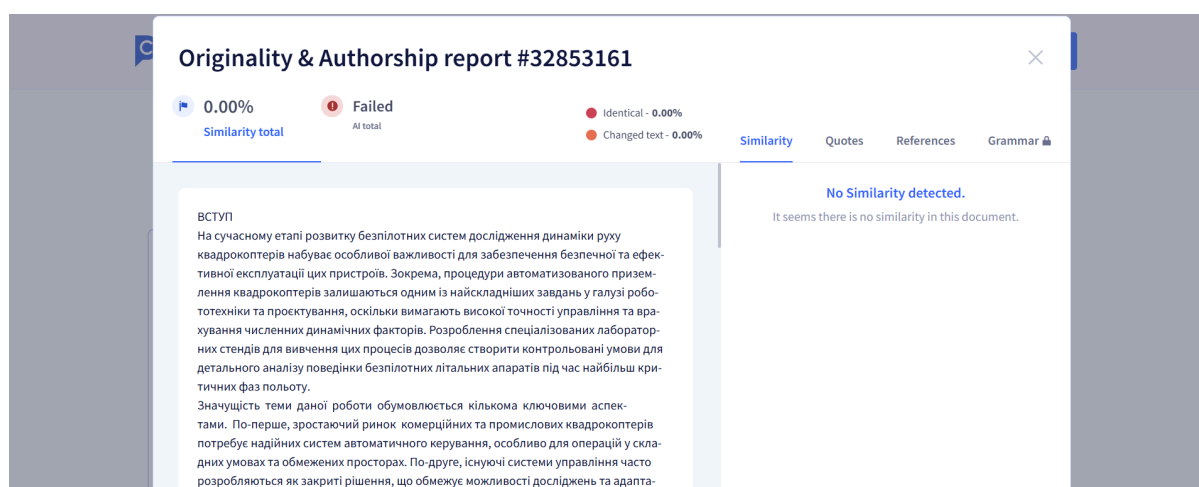


Рисунок 2.4 – Меню звіту в системі Plagiarismcheck.org

Звісно, як і попередній аналог, даний сервіс має підписочну бізнес-модель, тобто оплату за більшу кількість сторінок тексту.

Загалом, онлайн-сервіс Plagiarismcheck.org інтегрується з навчальними платформами, такими як Moodle, Google Classroom і т.п.; визначає контент, згенерований штучним інтелектом (варто зазначити, що ця функція працює лише з англійською та іспанською мовами); перевіряє авторство, порівнюючи документ з попередніми роботами студента. Всі ці функції роблять сервіс зручним для використання у навчальних закладах студентами та викладачами.

В той же час, сервіс має такі ж обмеження, як і попередній аналог, а саме: примітивний семантичний аналіз та неможливість використовувати всі можливості без платної підписки.

2.3 Програмне рішення «Grammarly»

Grammarly – це онлайн-платформа на основі штучного інтелекту для покращення продуктивності і комунікації. Серед продуктів, які представлені на цій платформі, присутня також і перевірка на плагіат. Цей інструмент перевірки на плагіат працює на базі штучного інтелекту та дозволяє порівняти текстовий документ з мільярдом веб-сторінок одним кліком, даючи оцінку унікальності вашого документа [3].

Для початку роботи з сервісом потрібно просто завантажити файл або ввести текст в призначене для цього поле. Цей функціонал зображено на рисунку 2.5.

Plagiarism Checker

Ensure every word is your own with Grammarly's AI-powered plagiarism checker, which uses advanced AI to detect plagiarism in your text and check for other writing issues.

Type, paste, or upload your text.

Scan for plagiarism Upload file Try sample text

Plagiarism Checker results

Add your text and click Scan for plagiarism to see your results.

No plagiarism found	Grammar
Spelling	Punctuation
Conciseness	Readability
Word choice	Additional issues

Рисунок 2.5 – Меню завантаження текстового документу в системі Grammarly

Після завантаження текстового документу користувач може натиснути на кнопку «Scan for plagiarism», що запустить процес аналізу. В результаті користувач отримає звіт, в якому вказано відсоток збігів, а також виділено фрагменти тексту, що співпадають з іншими джерелами, і вказано посилання на ці джерела. Окрім цього, пропонуються рекомендації щодо покращення граматики, пунктуації та інших помилок, знайдених у тексті. Цей функціонал зображено на рисунку 2.6.

Plagiarism Checker

Ensure every word is your own with Grammarly's AI-powered plagiarism checker, which uses advanced AI to detect plagiarism in your text and check for other writing issues.

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ «КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО» Факультет інформатики та обчислювальної техніки Кафедра інформаційних систем та технологій До захисту допущено: Завідувач кафедри _____ Олександр РОЛІК «___» _____ 2025 р. Дипломний проєкт на здобуття ступеня бакалавра за освітньо-професійною програмою «Інформаційне забезпечення робототехнічних систем» спеціальності 126 «Інформаційні системи та технології» на тему: «Лабораторний стенд для дослідження динаміки руху квадрокоптера» Виконав: студент IV курсу, групи ІК-12 Рябінін Роман Віталійович _____ Керівник: асистент каф. ІСТ Тюляков Дмитро Ігорович _____ Рецензент: доцент каф. ММСА, ННІПСА, к.т.н., доцент Кувєда Юлія Валеріївна

Scan first 1,500 words Upload file 15500/1,500 words

Plagiarism Checker results

28 We didn't find any plagiarism, but we found 28 writing issues.

No plagiarism found	Grammar
Spelling	Punctuation
Conciseness	Readability
Word choice	Additional issues

The first 1,500 words were scanned. Sign up to get more detailed feedback on your text. Get Grammarly It's free

Рисунок 2.6. – Меню звіту в системі Grammarly

Як і попередні аналоги, даний сервіс працює за підписочною бізнес-моделлю: базові функції доступні безкоштовно, але детальні звіти з вказанням джерел доступні тільки користувачам, що оформили платну підписку.

Загалом, онлайн-сервіс Grammarly вирізняється використанням сучасних алгоритмів обробки природньої мови, а також інтеграцією з текстовими редакторами, такими як Microsoft Word та Google Docs. Окрім цього, сервіс пропонує функціонал для аналізу читабельності текстів, що важливо для щоденної роботи.

В той же час, сервіс має такі ж обмеження, як і попередні аналоги, тобто семантичний аналіз реалізований лише частково і працює тільки на окремих перефразованих фрагментах, а також повний доступ до функціоналу закритий за платною підпискою. Варто також зазначити, що продукт орієнтований більше на англійську мову, що робить його нерелевантним при перевірці документів іншою мовою.

Висновок до розділу 2

У даному розділі було проведено аналіз трьох існуючих рішень для перевірки текстових документів на унікальність: Uniqecheck, Plagiarismcheck.org та Grammarly. Дані рішення широко застосовуються у сферах освіти, науки та бізнесу, даючи користувачам можливість оцінити документ на плагіат та побачити який відсоток тексту має запозичення з інших джерел.

Uniqecheck пропонує користувачу простий та зрозумілий інтерфейс для роботи з текстовими документами, забезпечує швидку перевірку тексту, а також використовує AI-алгоритми, які дозволяють виявити не лише дослівні збіги, а й випадки перефразування. Plagiarismcheck.org інтегрується з навчальними системами, а також пропонує інструменти для

визначення AI-контенту та перевірки авторства, що перетворює його в зручний сервіс для шкіл та університетів. Grammarly має в своєму арсеналі найбільшу кількість функцій, включаючи перевірку орфографії, граматики, пунктуації та стилю, а також перевірку на плагіат, яка сканує текст на збіги з величезною кількістю робіт.

Попри наявні переваги, розглянуті сервіси мають спільні недоліки.

По-перше, семантичний аналіз у них реалізований тільки частково і не дозволяє виявляти всі випадки змістовної подібності частин тексту. По-друге, сервіси не дають можливості виконати аналіз на рівні фрагментів, щоб визначити конкретні запозичення, а не тільки вивести загальний відсоток схожості. Насамкінець, кожен з цих аналогів працює за підписочною бізнес-моделлю, що обмежує доступ до повного функціоналу для нового користувача та змушує його платити за це гроші.

Отже, проведений аналіз існуючих рішень дав змогу підтвердити актуальність та доцільність розробки власної інформаційної системи, що буде усувати зазначені недоліки. Розроблювана система матиме функцію семантичного пошуку, яка працюватиме на основі векторних представлень, а також розділятиме текстовий документ на фрагменти та аналізуватиме кожен з них. Окрім цього, проєкт має власну базу даних, яку можна постійно доповнювати, а також використовує pgvector для полегшення семантичного пошуку. Все це в результаті дає нам швидкий, точний та дуже гнучкий інструмент для аналізу текстових документів та пошуку схожих матеріалів.

3 ПРОЄКТУВАННЯ АРХІТЕКТУРИ СИСТЕМИ

У даному розділі розглядається процес проєктування архітектури системи перевірки плагіату, що базується на комплексному аналізі текстових документів із використанням лексичних, структурних та

семантичних методів. Архітектура системи визначає загальну організацію її компонентів, принципи їхньої взаємодії, а також розподіл відповідальності між клієнтською, серверною частинами та підсистемою обробки даних.

Особлива увага приділяється формулюванню функціональних і нефункціональних вимог до системи, які визначають її основні можливості, обмеження та критерії якості. На основі цих вимог здійснюється обґрунтований вибір технологій реалізації кожної компоненти, з урахуванням продуктивності, масштабованості та зручності інтеграції.

У межах розділу також розглядаються ключові технологічні рішення, що використовуються для реалізації серверної та клієнтської частин системи, моделі векторного представлення тексту, а також система управління базами даних. Такий підхід дозволяє сформувати цілісну архітектуру системи, яка забезпечує ефективну та точну перевірку текстів на наявність плагіату.

3.1 Формулювання вимог до системи

Розробка інформаційної системи перевірки плагіату потребує чіткого визначення функціональних та нефункціональних вимог, які забезпечують коректну роботу програмного продукту, високу точність аналізу текстів та зручність використання системи користувачами. Основним призначенням системи є автоматизоване виявлення текстових запозичень із використанням комбінованого підходу, що включає семантичний, лексичний та структурний аналіз тексту.

3.1.1 Функціональні вимоги

Функціональні вимоги описують основні можливості системи та перелік задач, які вона повинна виконувати:

- система повинна забезпечувати завантаження текстових документів для перевірки на наявність плагіату;
- система повинна виконувати попередню обробку тексту (розбиття на рівні частини);
- система повинна здійснювати лексичний аналіз текстів із використанням статистичних методів (n-грами);
- система повинна виконувати структурний аналіз документів на основі їхньої організації (речення, абзаци, довжина текстових елементів);
- система повинна виконувати семантичне порівняння текстів із використанням векторних представлень (embeddings);
- система повинна обчислювати інтегральний показник схожості документів на основі комбінації різних методів;
- система повинна зберігати документи та результати перевірки у базі даних;
- система повинна надавати користувачу результати перевірки у вигляді відсоткового значення плагіату та деталізованого звіту;
- система повинна забезпечувати можливість перегляду історії перевірок документів.

3.1.2 Нефункціональні вимоги

Нефункціональні вимоги визначають якісні характеристики роботи системи:

- система повинна забезпечувати обробку документів у прийнятний час навіть при великій кількості текстів у базі даних;
- система повинна підтримувати розширення обсягу даних без суттєвого зниження продуктивності;
- результати перевірки повинні забезпечувати високу точність виявлення плагіату за рахунок комбінованого підходу;
- система повинна коректно обробляти помилки введення та збої без втрати даних;
- забезпечення захисту даних користувачів та завантажених документів;
- можливість додавання нових алгоритмів аналізу без значної зміни архітектури системи;
- інтерфейс системи повинен бути інтуїтивно зрозумілим для кінцевого користувача.

3.2 Вибір технологій розробки

Вибір технологій розробки є важливим етапом створення системи перевірки плагіату, оскільки саме від обраного програмного стеку залежать продуктивність, масштабованість, швидкодія та зручність подальшого супроводу системи. При виборі технологій враховувалися особливості обробки текстових даних, необхідність виконання семантичного аналізу, робота з векторними представленнями тексту, а також потреба у створенні сучасного веб-інтерфейсу для взаємодії з користувачем.

Для реалізації системи було обрано набір технологій, що забезпечують ефективну роботу як серверної, так і клієнтської частини застосунку. Особлива увага приділялася підтримці алгоритмів машинного навчання, можливості інтеграції моделей обробки природної мови та зручній роботі з базами даних.

У даному підрозділі буде розглянуто основні технології, фреймворки та бібліотеки, обрані для розробки системи, а також обґрунтовано доцільність їхнього вибору відповідно до поставлених функціональних і нефункціональних вимог.

3.2.1 Система управління базами даних

Система управління базами даних (СУБД) є ключовим компонентом інформаційної системи перевірки плагіату, оскільки забезпечує зберігання, організацію та швидкий доступ до структурованих даних. У межах даного проєкту база даних використовується для збереження завантажених документів, результатів аналізу, проміжних обчислень, а також історії перевірок користувачів.

Основними вимогами до СУБД були висока продуктивність при роботі з текстовими даними, підтримка складних запитів, надійність зберігання інформації та можливість масштабування. Також важливим фактором була зручна інтеграція з серверною частиною системи та підтримка роботи з напівструктурованими даними.

У якості системи управління базою даних у проєкті буде використовуватися PostgreSQL. Дана СУБД була обрана завдяки її високій стабільності, відкритому коду та широким можливостям роботи з великими обсягами даних. PostgreSQL добре підходить для задач аналітики та складних вибірок, що є важливим у контексті аналізу схожості документів.

Додатковою перевагою PostgreSQL є підтримка розширень для роботи з векторними представленнями даних, зокрема можливість використання pgvector, що дозволяє ефективно зберігати та порівнювати embedding-вектори, отримані в процесі семантичного аналізу тексту. Це

суттєво підвищує продуктивність семантичного пошуку та порівняння документів у системі.

Отже, вибір PostgreSQL як СУБД забезпечує надійне зберігання даних, ефективну роботу з великими обсягами текстової інформації та підтримку сучасних методів векторного аналізу, що є критично важливим для системи перевірки плагіату.

3.2.2 Серверна частина

Серверна частина є основним виконавчим компонентом системи перевірки плагіату, оскільки саме вона відповідає за обробку текстових даних, виконання алгоритмів аналізу та формування фінального результату перевірки. Вона реалізує всю бізнес-логіку системи, включаючи лексичний, структурний та семантичний аналіз документів, а також взаємодію з базою даних.

До основних вимог до серверної частини належать висока продуктивність при обробці текстів, підтримка асинхронних запитів, можливість інтеграції з моделями машинного навчання та зручність розширення функціоналу. Окремо враховувалася необхідність ефективної роботи з векторними представленнями текстів та швидкого обчислення метрик схожості.

У якості основного інструменту для реалізації серверної частини було обрано Python як мову програмування, оскільки вона має широкий набір бібліотек для обробки природної мови та машинного навчання. Для побудови API буде використовуватися FastAPI, який забезпечує високу швидкість роботи, підтримку асинхронності та автоматичну генерацію документації (Swagger/OpenAPI), що спрощує тестування та інтеграцію системи.

Для реалізації алгоритмів аналізу тексту будуть використані бібліотеки для роботи з NLP та векторними представленнями, зокрема моделі SentenceTransformer, які дозволяють отримувати семантичні embeddings документів. Також серверна частина виконуватиме обчислення лексичної та структурної схожості, після чого формуватиме інтегральний показник плагіату.

Саме так серверна частина забезпечуватиме централізовану обробку даних, реалізацію всіх алгоритмів аналізу та взаємодію між клієнтською частиною і базою даних, що робить її ядром усієї системи перевірки плагіату.

3.2.3 Модель векторного перетворення тексту

Модель векторного перетворення тексту є центральним компонентом системи перевірки плагіату, оскільки саме вона забезпечує можливість семантичного аналізу документів. Основне її призначення полягає у перетворенні текстових даних у числові векторні представлення (embeddings), які відображають зміст тексту у багатовимірному просторі та дозволяють виконувати математичне порівняння документів.

На відміну від класичних підходів, таких як bag-of-words або TF-IDF, векторні моделі враховують контекст слів, їхні взаємозв'язки та семантичне значення, що дозволяє виявляти не лише прямі збіги, але й перефразований або переформульований текст.

У процесі роботи системи текст спочатку буде розбиватися на менші фрагменти (chunks), після чого кожен фрагмент буде окремо перетворено у вектор. Далі отримані вектори агрегуються для формування загального представлення документа.

У даній системі буде використовуватися модель SentenceTransformer("all-MiniLM-L6-v2"), яка належить до сімейства

трансформерних моделей для побудови sentence embeddings. Вона є попередньо навченою нейронною мережею, побудованою на основі архітектури трансформерів та спеціально оптимізованою для задач порівняння текстових фрагментів.

У сфері семантичного аналізу тексту існує декілька альтернативних підходів і моделей, зокрема Word2Vec та GloVe, які формують векторні представлення слів і потребують додаткового усереднення для отримання вектора всього документа. Також використовується модель Doc2Vec, яка створює представлення одразу для документів, однак її якість сильно залежить від навчального корпусу та часто є нестабільною на різнорідних даних. Більш сучасними є трансформерні моделі, такі як BERT та його модифікації, які забезпечують глибоке контекстне розуміння тексту, але при цьому є значно важчими з точки зору обчислювальних ресурсів. Окремо варто відзначити моделі сімейства Sentence-BERT (SBERT), які спеціально адаптовані для задач порівняння текстів і є найближчими за концепцією до обраної моделі.

Вибір саме all-MiniLM-L6-v2 обумовлений тим, що вона є оптимізованим варіантом трансформерної архітектури, який поєднує високу якість семантичного представлення з низькою обчислювальною складністю. У порівнянні з повноцінними моделями BERT або більшими SBERT-варіантами, вона значно швидша та менш ресурсомістка, що є критично важливим для системи перевірки плагіату, де необхідно обробляти велику кількість документів у реальному часі. При цьому точність визначення семантичної схожості залишається на високому рівні, достатньому для виявлення перефразованого та змістовно подібного тексту.

Додатковою перевагою є її хороша узгодженість з метрикою cosine similarity, що спрощує інтеграцію у систему та дозволяє ефективно порівнювати векторні представлення документів без складних додаткових

перетворень. Таким чином, обрана модель забезпечує найкращий баланс між швидкістю, точністю та практичною придатністю для задачі виявлення плагіату.

3.2.3 Клієнтська частина

Клієнтська частина є важливим елементом системи перевірки плагіату, оскільки забезпечує взаємодію користувача з основним функціоналом системи. Вона відповідає за формування інтерфейсу користувача, прийом і передачу вхідних даних на сервер, а також відображення результатів аналізу у зручному та зрозумілому вигляді.

Основними вимогами до клієнтської частини є інтуїтивність інтерфейсу, швидкість роботи, коректна взаємодія з серверною частиною через REST API, а також можливість зручного завантаження текстових документів і перегляду результатів перевірки. Важливим аспектом також є візуалізація результатів аналізу у вигляді відсоткового показника схожості та детального звіту по кожному етапу перевірки.

У даній системі клієнтська частина реалізована з використанням бібліотеки React 18.3.1, яка дозволяє створювати компонентний та реактивний інтерфейс користувача. Для швидкого запуску та оптимізації збірки проєкту використовується Vite, що забезпечує високу швидкість розробки та миттєве оновлення змін під час роботи. Для стилізації інтерфейсу застосовується Tailwind CSS, який дозволяє будувати адаптивний та сучасний дизайн без необхідності написання великої кількості кастомних стилів.

Взаємодія клієнтської частини із сервером здійснюється через HTTP-запити до REST API, що забезпечує чітке розділення фронтенд- та бекенд-логіки. Клієнтська частина надсилає текстові дані для аналізу,

отримує результати обчислень та відображає їх у структурованому вигляді користувачу.

Таким чином, клієнтська частина забезпечує зручний та інтуїтивний інтерфейс для роботи із системою перевірки плагіату, виконуючи роль проміжного шару між користувачем і серверною логікою системи.

Висновок до розділу 3

У даному розділі було здійснено проєктування архітектури системи перевірки плагіату, визначено її основні компоненти та принципи взаємодії між ними. Було сформульовано функціональні та нефункціональні вимоги до системи, які задають її ключові можливості, обмеження та критерії якості, зокрема продуктивність, масштабованість, точність та зручність використання.

На основі сформульованих вимог було обґрунтовано та обрано сучасний технологічний стек для реалізації кожної складової системи. Зокрема, визначено інструменти для реалізації серверної та клієнтської частин, систему управління базами даних, а також модель векторного представлення тексту для семантичного аналізу.

Таким чином, у межах розділу було закладено архітектурну основу системи, що поєднує сучасні підходи до обробки природної мови, веб-розробки та роботи з даними, що забезпечує ефективність і розширюваність розробленого рішення.

4 МАТЕМАТИЧНЕ ЗАБЕЗПЕЧЕННЯ СИСТЕМИ

У даному розділі розглядається математичне забезпечення системи перевірки плагіату, яке є основою для кількісного оцінювання ступеня схожості текстових документів. Оскільки задача виявлення плагіату не

може бути вирішена одним універсальним методом, система використовує комплексний підхід, що поєднує декілька взаємодоповнюючих способів аналізу тексту.

Зокрема, буде детально описано три основні види оцінки подібності, які застосовуються в системі:

- лексичний аналіз, що базується на статистичних характеристиках слів;
- структурний аналіз, який враховує організацію та побудову тексту;
- семантичний аналіз, що дозволяє оцінювати змістову схожість документів за допомогою векторних подань.

Крім того, буде наведено математичні моделі та формули, що використовуються для обчислення кожного виду подібності, а також описано підхід до інтеграції отриманих результатів у єдиний узагальнений показник плагіату. Такий підхід забезпечує більш точну та об'єктивну оцінку, оскільки враховує як поверхневі, так і глибинні характеристики тексту.

4.1 Лексичне порівняння

Лексичне порівняння є одним із базових етапів оцінювання схожості текстових документів у системі перевірки плагіату. Його основна ідея полягає у представленні тексту як набору слів або послідовностей слів та подальшому статистичному аналізі їхнього збігу. Даний підхід дозволяє виявляти прямі або часткові запозичення текстових фрагментів без глибокого аналізу їхнього змісту.

Для подальшого математичного аналізу кожен документ перетворюється у вектор ознак. Найчастіше для цього використовується модель **TF-IDF (Term Frequency – Inverse Document Frequency)**, яка дозволяє оцінити важливість кожного слова в межах документа відносно

всієї колекції текстів. У даному контексті терм (term) – це базова одиниця тексту, яка використовується для аналізу і може відповідати окремому слову або послідовності слів (n-грамі), залежно від обраної моделі представлення. Саме для кожного такого терма і виконується розрахунок значень TF та IDF, що дозволяє визначити його значущість у межах документа та всього корпусу текстів.

Значення TF-IDF для терма t у документі d обчислюється як:

$$TFIDF(t, d) = TF(t, d) \cdot IDF(t)$$

де:

$TF(t, d)$ – частота входження терма у документі;

$$IDF(t) = \log \frac{N}{DF(t)} ;$$

N – загальна кількість документів у корпусі;

$DF(t)$ – кількість документів, що містять терм t .

Таким чином, рідкісні, але інформативні слова отримують більшу вагу, ніж часто вживані.

Окрім окремих слів, у системі також можуть використовуватися n-грамні моделі (біграми, триграми), які враховують послідовності з кількох слів. Це дозволяє частково зберігати локальний порядок слів і підвищує точність виявлення фрагментарного копіювання тексту.

Після побудови векторних представлень документів застосовується міра схожості, найчастіше – **косинусна подібність**:

$$sim(A, B) = \frac{A \cdot B}{\|A\| \cdot \|B\|}$$

де:

A і B – TF-IDF вектори двох документів,

$A \cdot B$ – скалярний добуток векторів,

$\|A\|$, $\|B\|$ – їхні норми.

Отримане значення знаходиться в діапазоні від 0 до 1, де 1 означає повний збіг текстів.

Лексичне порівняння є швидким та ефективним методом попереднього аналізу текстів. Воно дозволяє виявляти явні збіги між документами та формує базовий рівень оцінки схожості, який у подальшому доповнюється структурним і семантичним аналізом для підвищення точності системи.

4.2 Структурне порівняння

Структурне порівняння є важливим компонентом системи перевірки плагіату, який дозволяє аналізувати не лише зміст тексту, але й його організаційну та синтаксичну побудову. На відміну від лексичного підходу, де основною одиницею є слово або n-грама, структурний аналіз розглядає документ як послідовність елементів вищого рівня – речень, абзаців та їхніх характеристик.

Основна мета структурного порівняння полягає у виявленні подібності між документами за рахунок однакового або схожого розташування текстових елементів, навіть якщо окремі слова були замінені синонімами або перефразовані.

Для кожного документа формується вектор структурних ознак, який може включати такі характеристики:

- кількість речень у документі;
- середня довжина речень (у словах або символах);
- кількість абзаців;
- середня довжина абзаців;
- розподіл довгих і коротких речень;
- порядок розташування речень або їхніх типів (розповідні, питальні тощо).

Таким чином, документ подається як структурований набір числових параметрів:

$$D = (f_1, f_2, f_3, \dots, f_n)$$

де f_i – окрема структурна ознака документа.

Після побудови векторів структурних ознак для двох документів застосовуються методи оцінки відстані або подібності. Найчастіше використовуються такі методи:

- косинусна подібність для нормалізованих векторів;
- **евклідова відстань** для оцінки різниці між структурними характеристиками:

$$d(A, B) = \sqrt{\sum_{i=1}^n (A_i - B_i)^2}$$

де A_i, B_i – відповідні структурні ознаки двох документів.

Чим менше значення відстані або чим вища косинусна подібність, тим більш схожими є структури текстів.

Структурне порівняння є стійким до поверхневих змін тексту, таких як заміна слів або перефразування. Воно дозволяє виявляти випадки плагіату, коли збережена загальна логіка та побудова документа, але змінено лексичне наповнення.

Однак цей підхід сам по собі не є достатнім для повного виявлення плагіату, оскільки різні тексти можуть мати подібну структуру випадково. Тому він використовується у поєднанні з лексичним та семантичним аналізом для підвищення точності системи.

4.3 Семантичне порівняння

Семантичне порівняння є найбільш сучасним та змістовно глибоким методом аналізу текстів у системі перевірки плагіату. На відміну від лексичного та структурного підходів, які працюють із поверхневими характеристиками тексту (слова, n-грами, структура речень), семантичний аналіз спрямований на визначення змістової схожості документів незалежно від форми їхнього представлення.

Основна ідея полягає в тому, щоб представити текст у вигляді щільного числового вектора (embedding), який відображає його зміст у багатовимірному семантичному просторі.

Перед побудовою векторного представлення текст попередньо розбивається на менші фрагменти (chunks). Це дозволяє краще враховувати довгі документи та локальні змістові залежності, після чого кожен фрагмент окремо перетворюється у векторне подання, а результуючий вектор документа формується шляхом агрегації отриманих представлень.

У сучасних системах існує кілька основних підходів до формування embedding-представлень тексту:

1. Word2Vec / GloVe – генерують вектори для окремих слів. Для отримання представлення документа потрібно додатково усереднювати вектори слів. Недоліком є слабе врахування контексту та порядку слів.

2. Doc2Vec – формує вектор одразу для всього документа. Однак якість представлення сильно залежить від навчання на конкретному корпусі та може бути нестабільною на різнорідних текстах.

3. Transformer-based моделі (BERT, Sentence-BERT та похідні) – використовують механізм self-attention, що дозволяє враховувати контекст у межах усього речення або тексту. Забезпечують значно кращу якість семантичного представлення.

Серед трансформерних моделей особливою популярністю користуються sentence-embedding моделі, зокрема сімейство **Sentence-BERT**, які оптимізовані саме для задач порівняння текстів.

Для даної системи було обрано модель **SentenceTransformer ("all-MiniLM-L6-v2")**, яка є легкою та ефективною версією трансформерної архітектури.

Дана модель була обрана з наступних причин:

- висока швидкість обчислення, що є критично важливим для системи перевірки плагіату з великою кількістю документів;
- оптимальний баланс між точністю та продуктивністю – модель забезпечує якісні семантичні представлення при помірних обчислювальних витратах;
- хороша узагальнюваність, оскільки модель попередньо навчена на великому корпусі текстів і ефективно працює з різними типами даних;
- підтримка cosine similarity, що спрощує інтеграцію у систему оцінювання схожості.

Після обробки тексту модель перетворює його у щільний вектор:

$$D = (x_1, x_2, x_3, \dots, x_n)$$

де x_i – компоненти семантичного представлення, що відображають змістові характеристики тексту.

Далі схожість між документами визначається за допомогою косинусної подібності.

Отже, семантичне порівняння дозволяє виявляти глибинну змістову схожість текстів навіть у випадках суттєвого перефразування або зміни лексичної структури. Використання моделі SentenceTransformer ("all-MiniLM-L6-v2") забезпечує ефективний баланс між точністю та

продуктивністю, що робить її оптимальним вибором для даної системи перевірки плагіату.

Висновок до розділу 4

У даному розділі було розглянуто математичне забезпечення системи перевірки плагіату, яке є основою для кількісної оцінки ступеня схожості текстових документів. Детально описано три основні підходи до аналізу тексту: лексичне, структурне та семантичне порівняння.

Лексичне порівняння базується на n-грамному аналізі та дозволяє виявляти прямі збіги текстових фрагментів. Структурне порівняння враховує організацію тексту, його побудову та статистичні характеристики, що дозволяє оцінювати схожість навіть при зміні лексичного наповнення. Семантичне порівняння, у свою чергу, забезпечує аналіз змістової подібності документів за допомогою векторних представлень тексту, що дозволяє виявляти перефразований або частково змінений контент.

Поєднання цих підходів забезпечує комплексний аналіз текстових документів, підвищує точність визначення плагіату та дозволяє враховувати як поверхневі, так і глибинні характеристики тексту. Таким чином, розроблене математичне забезпечення є достатнім для ефективного функціонування системи перевірки плагіату.

5 РОЗРОБЛЕННЯ ІНФОРМАЦІЙНОЇ СИСТЕМИ

5.1 Створення структури бази даних

У межах системи база даних використовується для збереження завантажених документів, розбитих фрагментів тексту (chunks), векторних представлень (embeddings), а також результатів аналізу та пошукових

запитів. Окремо зберігається інформація про ступінь схожості між документами, що дозволяє формувати історію перевірок і забезпечує можливість повторного використання вже обчислених даних.

Структура бази даних була спроектована таким чином, щоб підтримувати основні функції системи, зокрема: завантаження та зберігання документів, їх попередню обробку, семантичне та лексичне порівняння текстів, а також швидкий пошук схожих фрагментів. Особлива увага приділялася можливості ефективної роботи з великими обсягами текстових даних і підтримці векторного представлення інформації для семантичного аналізу.

Таким чином, база даних виконує роль основного сховища інформації та забезпечує взаємодію між усіма компонентами системи, що дозволяє реалізувати повний цикл перевірки плагіату – від завантаження документа до отримання фінального результату.

Поле	Тип даних	Опис
id	integer / UUID	Унікальний ідентифікатор документа в системі
title	text	Назва документа, яку надає користувач
content	text	Повний текст документа, що підлягає аналізу
created_at	timestamp	Дата та час створення/завантаження документа
hash	text	Хеш документа для швидкого визначення дублювання або повторного завантаження
total_characters	integer	Загальна кількість символів у документі

total_pages	integer	Орієнтовна кількість сторінок документа
total_chunks	integer	Кількість фрагментів (chunk'ів), на які розбитий документ
status	text / enum	Статус обробки документа (наприклад: uploaded, processing, completed, failed)

Таблиця 5.1 – Documents (основна таблиця документів)

Поле	Тип даних	Опис
id	integer / UUID	Унікальний ідентифікатор фрагмента
document_id	integer / UUID	Зовнішній ключ на таблицю documents
chunk_index	integer	Порядковий номер фрагмента в документі
text	text	Текстовий вміст конкретного фрагмента
char_count	integer	Кількість символів у фрагменті
created_at	timestamp	Дата створення фрагмента

Таблиця 5.2 – Chunks (фрагменти документа)

Поле	Тип даних	Опис
id	integer / UUID	Унікальний ідентифікатор embedding'у
document_id	integer / UUID	Ідентифікатор документа
embedding	vector / float[]	Векторне представлення тексту (SentenceTransformer)
created_at	timestamp	Дата створення embedding'у

chunk_id	integer / UUID	Посилання на відповідний chunk (якщо embedding створений для фрагмента)
metadata	json / text	Додаткова інформація (модель, параметри, версія обробки тощо)

Таблиця 5.3 – Embeddings (векторні представлення)

Поле	Тип даних	Опис
id	integer / UUID	Унікальний ідентифікатор запиту
query_text	text	Текст пошукового/перевірочного запиту
query_embedding	vector / float[]	Векторне представлення запиту
created_at	timestamp	Дата створення запиту

Таблиця 5.4 – Search_queries (запити користувача)

Поле	Тип даних	Опис
id	integer / UUID	Унікальний ідентифікатор запису
query_id	integer / UUID	Зовнішній ключ на search_queries
chunk_index	integer	Порядковий номер фрагмента запиту
text	text	Текстова частина запиту
char_count	integer	Кількість символів у фрагменті
created_at	timestamp	Дата створення
embedding	vector / float[]	Векторне представлення фрагмента запиту
start_page	integer	Початкова сторінка (для прив'язки до документа)

end_page	integer	Кінцева сторінка
----------	---------	------------------

Таблиця 5.5 – Search_query_chunks (розбиті частини запиту)

Поле	Тип даних	Опис
id	integer / UUID	Унікальний ідентифікатор результату
query_id	integer / UUID	Ідентифікатор запиту
matched_document_id	integer / UUID	Документ, який має збіг
score	float	Рівень схожості (0–1 або 0–100%)
rank	integer	Ранжування результату серед інших збігів
created_at	timestamp	Дата формування результату
metadata	json / text	Додаткова інформація (тип збігу, метод, ваги тощо)
matched_chunk_id	integer / UUID	Конкретний фрагмент документа, де знайдено збіг

Таблиця 5.6 – Search_results (результати пошуку/плагіату)

5.2 Розроблення серверної частини

Серверна частина системи перевірки плагіату була реалізована як фундаментальний компонент, що відповідає за обробку всіх бізнес-процесів, аналіз текстових документів та взаємодію з базою даних. Основною метою її розробки було забезпечення швидкої, масштабованої та структурованої обробки запитів від клієнтської частини системи.

default		^
GET	/ Root	▼
GET	/documents Get Documents	▼
GET	/documents/stats Get Dashboard Stals	▼
GET	/documents/recent Get Recent Documents	▼
GET	/documents/{document_id} Get Document	▼
POST	/upload Upload File	▼
Search		^
GET	/search/recent Get Recent Checks	▼
GET	/search/results Get Search Results	▼
GET	/search/chart Get Chart Data	▼
Plagiarism		^
GET	/plagiarism/{document_id}/matches Get Similarity Matches	▼
GET	/plagiarism/{document_id} Get Document Detail	▼
GET	/plagiarism/{document_id}/chunks Get Document Chunks	▼

Рисунок 5.1 – Swagger

Для реалізації серверної логіки було використано фреймворк FastAPI, який дозволяє будувати високопродуктивні REST API із підтримкою асинхронної обробки запитів. Важливою перевагою FastAPI є автоматична генерація документації через Swagger (OpenAPI), що значно спростило тестування та інтеграцію API. Завдяки Swagger користувач може візуально перевіряти всі доступні ендпоінти, параметри запитів та формат відповідей без необхідності використання додаткових інструментів.


```
def ingest_pdf(pdf_path):
    filename = os.path.basename(pdf_path)

    # 1. extract + structured preprocessing
    pdf_stats, chunks = process_pdf(pdf_path)

    text = "\n".join([c["text"] for c in chunks])
    doc_hash = generate_hash(text)

    conn = get_connection()
    cur = conn.cursor()

    # 2. insert document
    cur.execute("""
        INSERT INTO documents
        (title, content, hash, total_characters, total_pages, total_chunks)
        VALUES (%s, %s, %s, %s, %s, %s)
        RETURNING id
    """, (
        filename,
        text,
        doc_hash,
        pdf_stats["total_characters"],
        pdf_stats["total_pages"],
        pdf_stats["total_chunks"]
    ))
```

Рисунок 5.2 – Файл завантаження документів в базу даних ingest.py

Даний модуль реалізує процес завантаження PDF-документів у систему перевірки плагіату та виконує їх попередню обробку перед подальшим аналізом. Основним призначенням цього компонента є автоматичне розбиття документа на текстові фрагменти (chunks), генерація векторних представлень тексту та збереження всієї необхідної інформації у базі даних.

Робота модуля починається з отримання PDF-файлу та виділення його назви за допомогою функцій стандартної бібліотеки Python. Після цього викликається функція process_pdf(), яка здійснює структуровану обробку документа. На даному етапі текст PDF-файлу витягується та розбивається

на окремі chunks. Для кожного фрагмента додатково визначаються його характеристики, зокрема кількість символів, початкова та кінцева сторінки документа.

Після обробки всі chunks об'єднуються у суцільний текст документа, для якого генерується SHA-256 hash. Хешування використовується для унікальної ідентифікації документа та запобігання дублюванню даних у системі.

Далі модуль встановлює з'єднання з базою даних PostgreSQL та виконує запис основної інформації про документ у таблицю documents. До бази зберігаються назва документа, повний текст, hash-значення, кількість символів, кількість сторінок та кількість chunks.

Після цього для кожного текстового фрагмента генеруються семантичні embeddings за допомогою функції get_embeddings(). Отримані векторні представлення використовуються для подальшого семантичного аналізу та пошуку схожих фрагментів між документами.

```
from collections import Counter

def get_ngrams(text, n=3):
    words = text.lower().split()
    return Counter(tuple(words[i:i+n]) for i in range(len(words)-n+1))

def ngram_overlap(text1, text2, n=3):
    a = get_ngrams(text1, n)
    b = get_ngrams(text2, n)

    if not a or not b:
        return 0.0

    intersection = sum((a & b).values())
    total = sum(a.values())

    return intersection / total
```

Рисунок 5.3 – Файл підрахунку лексичної схожості lexical.py

Даний модуль реалізує лексичний аналіз тексту на основі n-грамного підходу та використовується для оцінки ступеня текстової схожості між документами. Основною функцією модуля є формування n-грам із тексту та обчислення коефіцієнта їх перекриття.

Функція `get_ngrams()` виконує попередню обробку тексту, переводячи його у нижній регістр та розбиваючи на окремі слова. Після цього формується набір n-грам — послідовностей із n слів, де за замовчуванням використовується значення $n=3$ (триграми). Для підрахунку кількості входжень кожної n-грами використовується структура `Counter`, що дозволяє ефективно виконувати статистичний аналіз тексту.

Функція `ngram_overlap()` використовується для визначення ступеня лексичної схожості між двома текстами. Спочатку для кожного тексту генеруються набори n-грам, після чого обчислюється кількість спільних елементів між ними. Результат визначається як відношення кількості спільних n-грам до загальної кількості n-грам першого тексту. Отримане значення знаходиться у межах від 0 до 1, де значення, близьке до 1, свідчить про високий рівень лексичного збігу між текстами.

Такий підхід дозволяє ефективно виявляти прямі текстові запозичення та часткові збіги між документами, що робить його важливим компонентом системи перевірки плагіату.

```
def sentence_similarity(text1, text2):
    import re

    s1 = [s.strip().lower() for s in re.split(r'[.!?]', text1) if s.strip()]
    s2 = [s.strip().lower() for s in re.split(r'[.!?]', text2) if s.strip()]

    if not s1 or not s2:
        return 0.0

    matches = 0

    for sent1 in s1:
        for sent2 in s2:
            if sent1 == sent2 or sent1 in sent2 or sent2 in sent1:
                matches += 1
                break

    return matches / len(s1)
```

Рисунок 5.4 – Файл підрахунку структурної схожості structure.py

Даний модуль реалізує структурний аналіз тексту на рівні речень та використовується для оцінки ступеня структурної схожості між документами. Основною функцією модуля є порівняння речень двох текстів з метою виявлення однакових або частково схожих фрагментів.

Функція `sentence_similarity()` спочатку виконує попередню обробку тексту: за допомогою регулярних виразів текст розбивається на окремі речення за символами крапки, знаку оклику та знаку питання. Далі кожне речення очищується від зайвих пробілів і переводиться у нижній регістр для уніфікації порівняння.

Після формування списків речень виконується їх попарне порівняння. Для кожного речення першого тексту перевіряється наявність повного або часткового збігу з реченнями другого тексту. Збіг вважається знайденим у випадку:

- повної рівності речень;

- входування одного речення в інше.

Кількість знайдених збігів накопичується, після чого обчислюється коефіцієнт структурної схожості як відношення кількості співпадінь до загальної кількості речень у першому тексті.

Отримане значення знаходиться у діапазоні від 0 до 1, де більше значення свідчить про вищий рівень структурної подібності між документами. Такий підхід дозволяє виявляти випадки копіювання або часткового перефразування речень навіть при незначних змінах тексту, що робить модуль важливою складовою системи перевірки плагіату.

```
# -----  
# VECTOR SEARCH  
# -----  
def search_similar_chunks(query_vector, cur, top_k=3, threshold=0.4):  
    cur.execute(  
        """  
        SELECT  
            e.document_id,  
            e.chunk_id,  
            c.text,  
            e.embedding <=> %s::vector AS distance  
        FROM embeddings e  
        JOIN chunks c ON c.id = e.chunk_id  
        ORDER BY distance ASC  
        LIMIT %s  
        """,  
        (query_vector.tolist(), top_k)  
    )  
  
    results = cur.fetchall()  
  
    return [  
        (r[0], r[1], r[2], r[3])  
        for r in results  
        if r[3] <= threshold  
    ]
```

Рисунок 5.5 – Файл пошуку схожих документів з застосуванням векторів
(embeddings) search.py

Даний модуль реалізує основний процес перевірки документів на наявність плагіату та поєднує всі ключові компоненти системи: обробку PDF-файлів, генерацію embeddings, векторний пошук, лексичне та структурне порівняння, а також обчислення фінального відсотка схожості. Саме цей файл виконує роль центрального конвеєра (pipeline) аналізу документів.

Функція `search_similar_chunks()` реалізує пошук найбільш схожих фрагментів тексту в базі даних. Для цього використовується оператор векторної відстані $\leq$, який дозволяє знаходити найближчі embeddings за cosine distance. Результати сортуються за ступенем схожості, а також додатково фільтруються за пороговим значенням `threshold`.

Після отримання найбільш схожих fragments система виконує багаторівневий аналіз. Для кожного знайденого збігу обчислюється:

- семантична схожість на основі embedding distance;
- лексична схожість за допомогою n-грамного аналізу (`ngram_overlap`);
- структурна схожість на рівні речень (`sentence_similarity`).

Далі всі отримані метрики передаються у функцію `compute_final_score()`, яка формує інтегральний показник схожості між текстами. Саме цей показник використовується як фінальний рівень плагіату для конкретного фрагмента.

Після завершення аналізу результати зберігаються у базі даних через функцію `save_search_to_db()`. У таблиці `search_queries` зберігається інформація про сам запит, у `search_query_chunks` — окремі chunks документа разом із embeddings, а у `search_results` — результати знайдених збігів та їхні показники схожості.

На завершальному етапі система виконує усунення дублікатів результатів та обчислює загальний відсоток плагіату документа як середнє значення всіх отриманих similarity scores. Таким чином, даний модуль

реалізує повний цикл аналізу документа — від зчитування PDF-файлу до формування фінального відсотка схожості та збереження результатів перевірки у базі даних.

5.3 Розроблення веб-інтерфейсу користувача

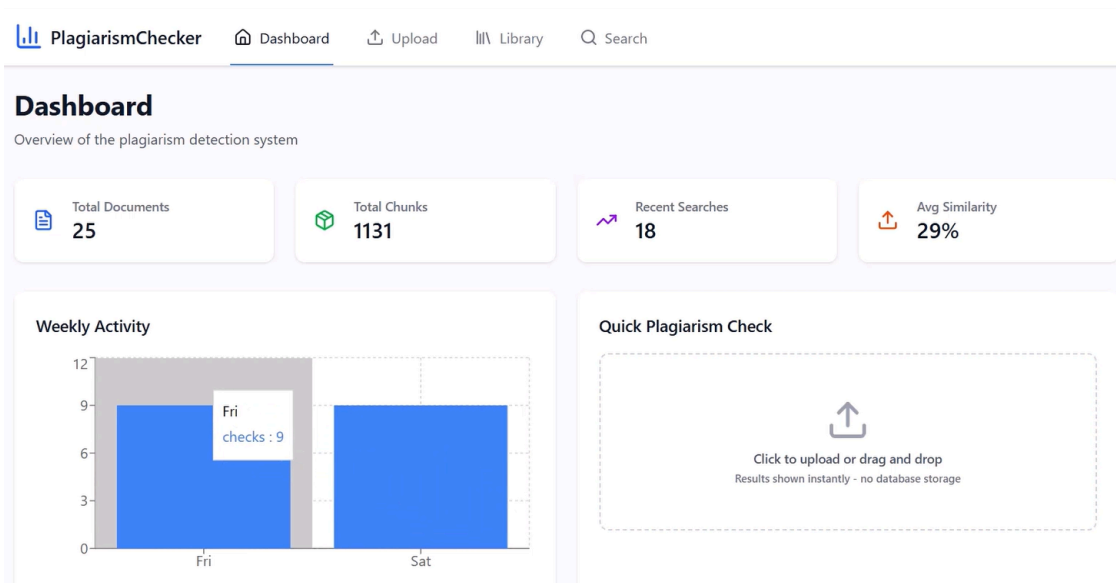


Рисунок 5.6 – Головна сторінка веб-додатку

Клієнтська частина системи містить основну сторінку у вигляді дашборду, яка виконує роль центрального екрана моніторингу роботи системи перевірки плагіату. Дашборд забезпечує користувачу швидкий доступ до ключової статистики та інформації про стан бази документів і результати останніх перевірок.

На головній сторінці відображається середній показник плагіату серед усіх перевірених документів, що дозволяє оцінювати загальний рівень схожості текстів у системі. Також дашборд містить інформацію про загальну кількість документів, збережених у базі даних, що дає змогу контролювати обсяг накопичених матеріалів для аналізу.

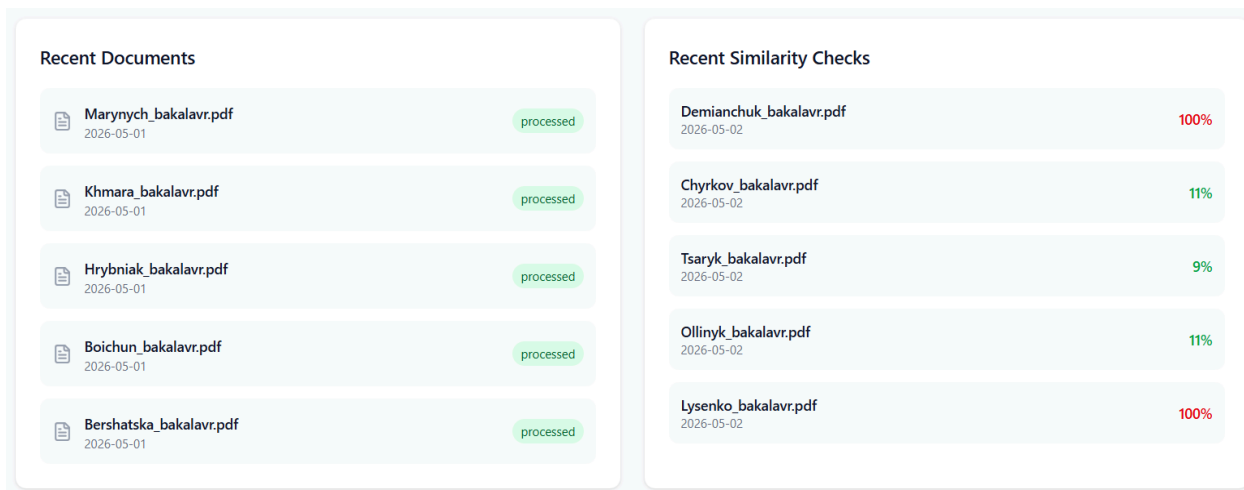


Рисунок 5.7 – Нещодавно завантажені та перевірені документи

Окремими блоками відображаються нещодавно завантажені документи та останні перевірені файли. Для кожного документа можуть виводитися його назва, дата завантаження та отриманий відсоток схожості. Такий підхід забезпечує зручний доступ до історії перевірок і дозволяє користувачу швидко переглядати результати останніх операцій без необхідності виконання додаткового пошуку.

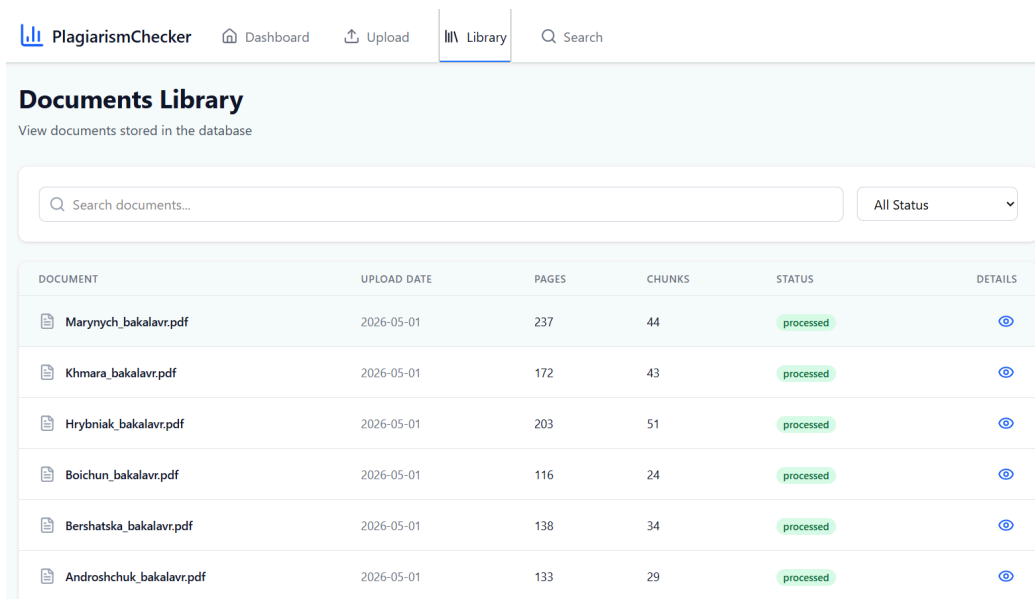


Рисунок 5.8 – Сторінка бібліотеки еталонних документів

Також клієнтська частина системи містить сторінку бібліотеки документів, яка забезпечує доступ до всіх файлів, збережених у базі даних. На цій сторінці відображається повний список завантажених документів із можливістю перегляду їх основної інформації, зокрема назви, дати завантаження та загальних характеристик. Бібліотека виконує роль центрального сховища інтерфейсу, де користувач може швидко знайти будь-який еталонний документ, з яким порівнюються нові завантажені тексти, та перейти до його детального аналізу.

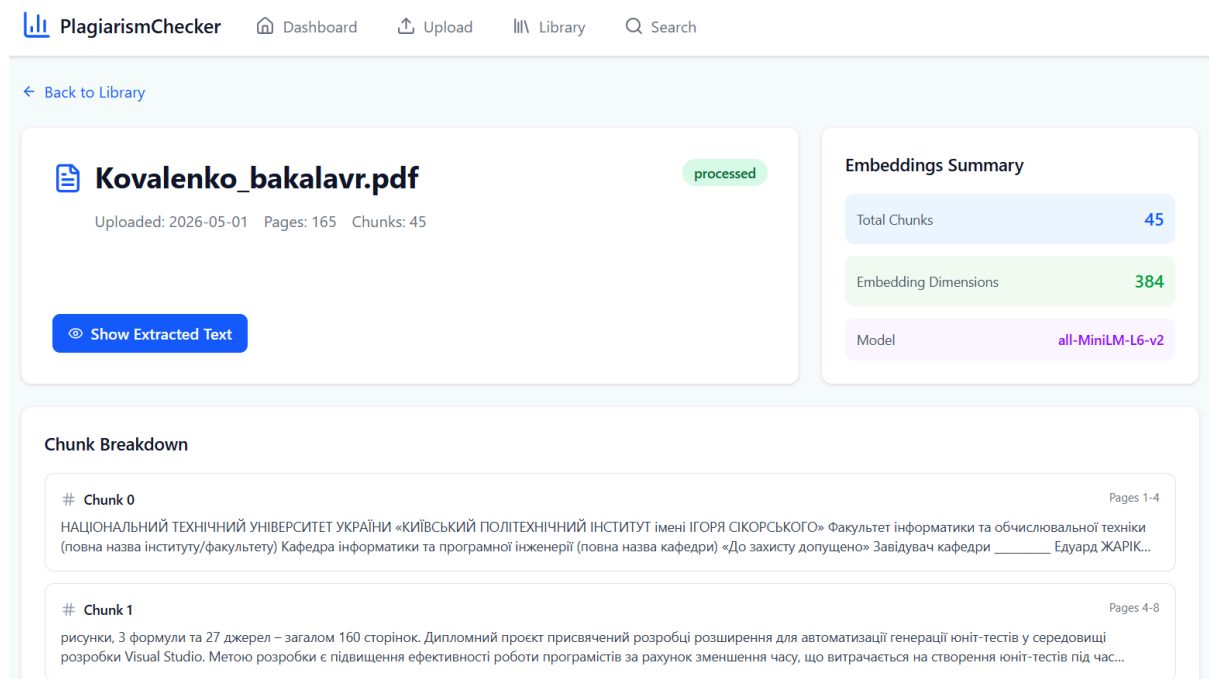


Рисунок 5.9 – Сторінка детальної інформації про еталонний документ

Окрім цього, реалізовано детальну сторінку документа, яка відкривається для кожного окремого файлу. На цій сторінці користувач має можливість переглянути повний текст документа, а також його структуру у вигляді окремих фрагментів (чанків), на які було розбито текст під час обробки. Для кожного фрагменту відображається його вміст, що дозволяє

детально аналізувати структуру документа та бачити, які саме частини тексту беруть участь у перевірці на плагіат.

Висновок до розділу 5

У результаті виконання цього розділу було розроблено повноцінну інформаційну систему перевірки плагіату, яка поєднує лексичний, структурний та семантичний підходи до аналізу текстових документів. Реалізована система забезпечує багаторівневу обробку даних, що дозволяє виявляти як прямі текстові збіги, так і перефразований або змістовно подібний контент.

Серверна частина системи реалізована з використанням FastAPI та забезпечує обробку документів, виконання алгоритмів аналізу та взаємодію з базою даних PostgreSQL. Для семантичного аналізу використовується модель SentenceTransformer("all-MiniLM-L6-v2"), яка дозволяє отримувати векторні представлення тексту та визначати змістову схожість документів.

Клієнтська частина розроблена на основі React 18.3.1 із використанням Vite та Tailwind CSS, що забезпечило сучасний, швидкий та адаптивний інтерфейс користувача. Інтерфейс системи включає дашборд зі статистикою, бібліотеку документів та детальні сторінки кожного файлу з можливістю перегляду їх структури та вмісту чанків.

База даних спроектована таким чином, щоб зберігати документи, їх фрагменти, векторні представлення та результати аналізу, що забезпечує ефективну організацію даних і можливість повторного використання вже обчислених результатів.

Таким чином, створена система є комплексним рішенням для автоматизованого аналізу текстових документів, яке поєднує сучасні методи обробки природної мови та веб-технології. Вона забезпечує високу

точність виявлення плагіату, зручність використання та можливість подальшого розширення функціоналу.

ВИСНОВКИ

Під час проходження переддипломної практики було розроблено сучасну інформаційну систему для аналізу текстових документів і пошуку схожих матеріалів. Основна мета системи — автоматизувати перевірку текстів на плагіат і визначати, наскільки документи схожі за змістом. Актуальність такої розробки пов'язана зі швидким зростанням кількості текстової інформації та потребою забезпечення академічної доброчесності в освіті, науці та бізнесі.

Система дозволяє завантажувати PDF-документи, виконувати їх попередню обробку, розбиття на фрагменти, створювати векторні представлення тексту та аналізувати схожість. Особливістю є комбінований підхід, який враховує лексичну, структурну та семантичну схожість. Це дає змогу знаходити не лише однакові фрагменти тексту, але й перефразований або змістовно подібний контент, що підвищує точність перевірки.

Під час практики було проаналізовано предметну область та сучасні сервіси перевірки плагіату, такі як Uniqecheck, PlagiarismCheck.org і Grammarly. Було визначено їхні переваги та недоліки, що допомогло обґрунтувати необхідність створення власної системи з більш глибоким аналізом тексту та роботою з фрагментами документів.

На етапі проєктування були визначені функціональні та нефункціональні вимоги, розроблена архітектура системи та обрано технології. Серверна частина реалізована на FastAPI та Python, що забезпечує швидку роботу, підтримку асинхронних запитів і зручну інтеграцію з алгоритмами машинного навчання. Клієнтська частина створена з використанням React, Vite і Tailwind CSS, що дозволило зробити інтерфейс сучасним та зручним.

Для збереження даних використано PostgreSQL з розширенням pgvector, яке дозволяє ефективно працювати з векторними представленнями тексту та швидко знаходити схожі документи. Також спроектовано базу даних для зберігання документів, векторів, результатів аналізу та історії перевірок.

Для створення семантичних векторів використано модель SentenceTransformer (“all-MiniLM-L6-v2”), яка добре поєднує швидкість і точність. Для визначення схожості застосовано cosine similarity, а також методи лексичного і структурного аналізу, що дозволило зробити перевірку більш комплексною.

На сервері реалізовано модулі для завантаження PDF-файлів, їх обробки, створення embeddings і збереження результатів у базі даних. Також розроблено REST API та автоматичну документацію Swagger, що спрощує тестування і використання системи.

У результаті створено систему, яка автоматично аналізує текстові документи, знаходить схожі матеріали та формує детальні результати перевірки. Архітектура системи є гнучкою і може бути розширена в майбутньому - наприклад, шляхом додавання нових алгоритмів, підтримки інших мов або інтеграції з навчальними платформами.

Загалом під час практики було закріплено теоретичні знання та отримано практичні навички у сфері розробки інформаційних систем, веб-технологій, обробки текстів і машинного навчання. Було продемонстровано вміння проходити всі етапи створення програмного продукту — від аналізу до реалізації та тестування. Створена система є корисним рішенням для автоматизації перевірки текстів на плагіат.

ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Plagiarism Statistics 2025: Facts, Trends, and Research Data [Електронний ресурс]. URL: <https://plagiarism-detector.com/en/plagiarism-statistics> (дата звернення: 01.05.2026).
2. Uniqecheck : онлайн-сервіс перевірки унікальності тексту [Електронний ресурс]. URL: <https://uniquecheck.com> (дата звернення: 01.05.2026).
3. PlagiarismCheck.org : онлайн-сервіс для перевірки на плагіат [Електронний ресурс]. URL: <https://plagiarismcheck.org> (дата звернення: 01.05.2026).
4. Grammarly : онлайн-платформа для виявлення плагіату та виправлення граматичних помилок [Електронний ресурс]. URL: <https://www.grammarly.com/plagiarism-checker> (дата звернення: 01.05.2026).